MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

REPORT

Laboratory work nr. 7-8

Subject: Operating Systems

Topic: “Processes, Redirection, Pipes (Linux & xv6)”

Elaborated: st. gr. FAF-243 Kushnirenko Ecaterina

Verified: asist. univ. Reiţman Patricia

Chişinău 2026

Contents

Introduction....3
1 Theoretical background....4
2 The shell's three steps: Linux (Laboratory 7)....5
2.1 fork + exec + wait....5
2.2 A zombie, on purpose....6
2.3 Redirection: ls -l > out.txt....7
2.4 A pipe: ls /etc | wc -l....7
3 The same calls, inside xv6 (Laboratory 8)....9
3.1 run, inside xv6....9
3.2 pingpong....10
Conclusions....12
Bibliography....13

Introduction

The objective of this laboratory work is to understand how processes are created and managed, how file descriptor redirection works, and how pipes connect processes — first on a real Linux system (Laboratory 7) and then inside xv6, a small teaching Unix (Laboratory 8). The work covers `fork`, `exec`, `wait`, zombie processes, file descriptor manipulation, and inter-process communication through pipes.

Laboratory work 7 (Linux): write and run four small C programs that demonstrate the three steps of the shell (`fork` + `exec` + `wait`), zombie processes, output redirection (`ls -l > out.txt`), and a pipe (`ls /etc | wc -l`). Record observations and answer the questions for each part.

Laboratory work 8 (xv6): port the fork + exec + wait program to xv6 and write a `pingpong` program that uses two pipes for parent–child communication. Compare the system calls between Linux and xv6.

Both labs were carried out on a personal laptop running Arch Linux (kernel 7.2.9-arch1-1, x86_64, 31 GiB of RAM, 1 TB NVMe SSD).

1 Theoretical background

A process is the operating system's unit of execution: a running program with its own address space, open file descriptors, and a position in the CPU scheduler's queue. In Unix, every process except the first (`init`, PID 1) is created by `fork()`, which duplicates the calling process. The child is an almost exact copy of the parent; the only difference visible to the program is the return value of `fork()`: zero in the child, the child's PID in the parent.

After `fork()`, the child typically calls `exec()` to replace its memory image with a new program. This is the fork-then-exec pattern that every Unix shell uses: fork to create a new process, exec to load the desired program into it, and `wait` (or `waitpid`) in the parent to collect the child's exit status. If the parent does not call wait, the terminated child becomes a **zombie** — it has finished executing but its entry in the process table is kept so the parent can later read its exit status.

File descriptors are small integers that refer to open files, pipes, or devices. Every process starts with three: 0 (`stdin`), 1 (`stdout`), and 2 (`stderr`). The shell implements output redirection (`command > file`) by closing descriptor 1 in the child and then opening the target file; because `open()` always returns the lowest available descriptor, it returns 1, so the program's `printf`/`write` calls go to the file without the program knowing.

A pipe is a kernel buffer with two file descriptors: one for reading and one for writing. The shell implements `cmd1 | cmd2` by creating a pipe, forking twice, and connecting `cmd1`'s stdout to the pipe's write end and `cmd2`'s stdin to the pipe's read end. When all writers close the write end, the reader gets EOF and can finish. If a writer never closes, the reader hangs.

xv6 implements the same system calls (`fork`, `exec`, `wait`, `pipe`, `read`, `write`) in about 9,000 lines of C for RISC-V. The API is intentionally similar to Unix so that programs like `run.c` require only minor changes (different headers, no glibc, `exec` instead of `execlp`) to run inside xv6.

2 The shell's three steps: Linux (Laboratory 7)

This chapter corresponds to Laboratory work 7. Four C programs were written, compiled, and run, each demonstrating a different OS mechanism.

2.1 fork + exec + wait

The program `run.c` does what the shell does when the user types a command: it calls `fork()` to create a child process, the child calls `execlp()` to become the target program, and the parent calls `waitpid()` to wait for the child to finish.

Three variants were tested: running `ls -l`, running `date`, and running a nonexistent program (`nosuchprog`).

Variant 1: ls -l. The child becomes `ls`, which lists the files in the working directory. The parent waits and prints the child's exit status (Figure 2.1).

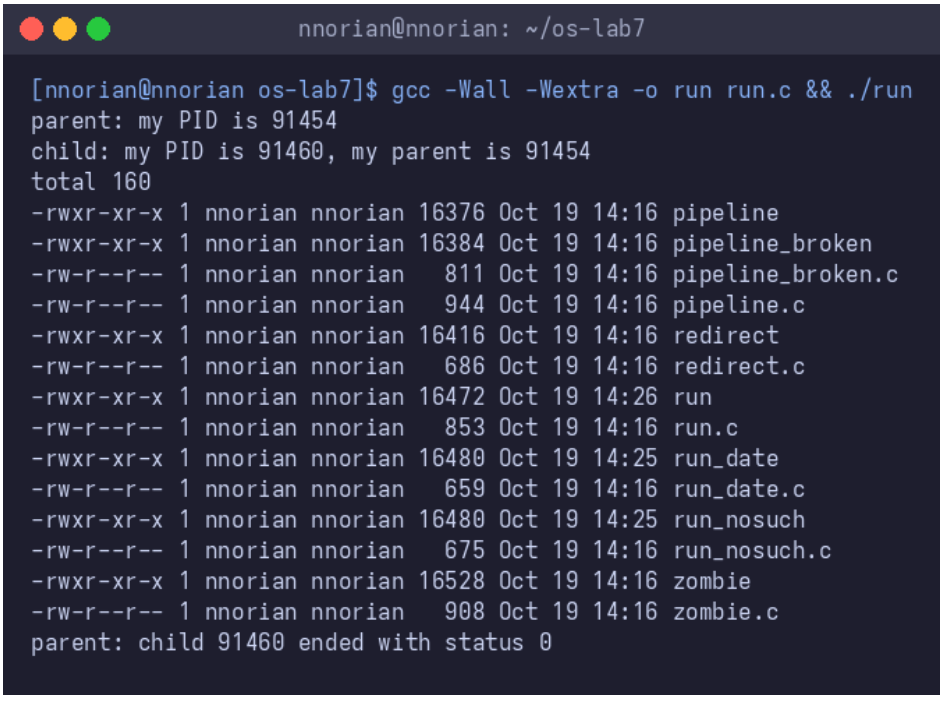


Figure 2.1 – run.c executing ls -l

Variant 2: date. The `execlp` line was changed to run `date` instead of `ls` (Figure 2.2).

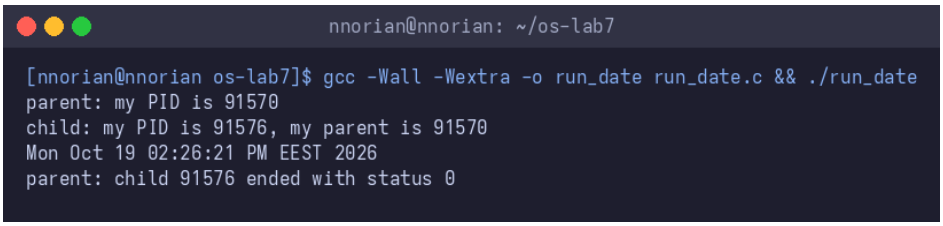


Figure 2.2 – run.c executing date

Variant 3: nosuchprog. The `execlp` line was changed to run a nonexistent program (Figure 2.3).

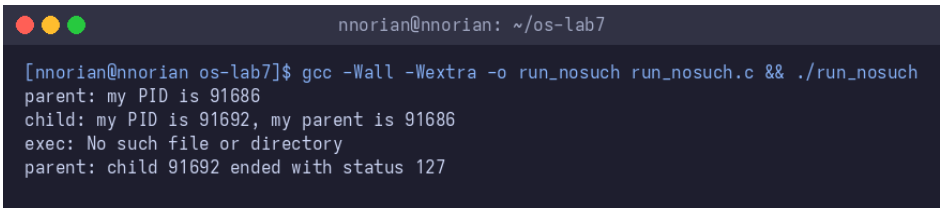


Figure 2.3 – run.c executing a nonexistent program

Observe answers:

1. The line `perror("exec")` is printed only with `nosuchprog` because `execlp()` only returns when it fails. When `execlp()` succeeds (as with `ls` or `date`), the child's entire memory image is replaced by the new program, so the code after `execlp()` — including `perror` — never runs. With `nosuchprog`, exec fails because the program does not exist, so execution continues to the next line and `perror` prints the error.

2. The child prints the parent's PID as "my parent". In the screenshot, the child says "my parent is 90750", which matches the parent's "my PID is 90750". This confirms that `getppid()` in the child returns the PID of the process that called `fork()`.

3. With `ls`, the parent sees status 0 (success). With `nosuchprog`, the parent sees status 127. The status 127 comes from the explicit `exit(127)` in the child after exec fails — this is the conventional Unix exit code for "command not found".

2.2 A zombie, on purpose

A zombie is a child process that has terminated but has not been waited for by its parent. The `zombie.c` program adds `sleep(20)` before the `waitpid()` call, creating a 20-second window during which the child (which runs `ls` and exits quickly) is a zombie.

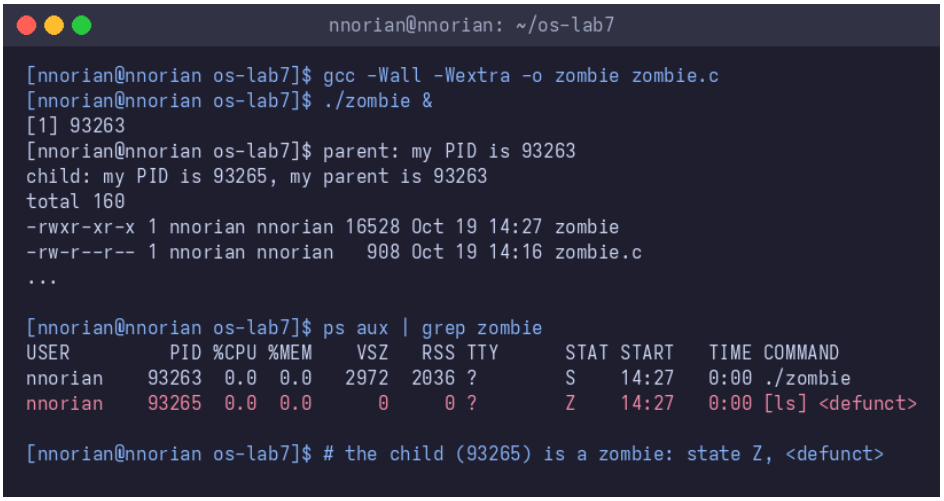


Figure 2.4 – The zombie process visible in ps output

The `ps` output (Figure 2.4) shows two processes. The parent (PID 93263) has state **S** (sleeping, in the `sleep(20)` call). The child (PID 93265) has state **Z** (zombie) and is shown as `[ls] <defunct>`. The child has finished executing `ls` and exited, but the parent has not yet called `waitpid()` to collect its exit status, so the kernel keeps the child's entry in the process table with zero memory (VSZ and RSS are both 0). After the parent wakes up and calls `waitpid()`, the zombie is reaped and disappears.

2.3 Redirection: ls -l > out.txt

The program `redirect.c` does what the shell does for output redirection. In the child, it closes file descriptor 1 (stdout), then opens `out.txt`. Because `open()` always returns the lowest available descriptor, it returns 1. After that, `execlp("ls", ...)` runs `ls`, which writes to descriptor 1 as always — but descriptor 1 now points to `out.txt` instead of the terminal.

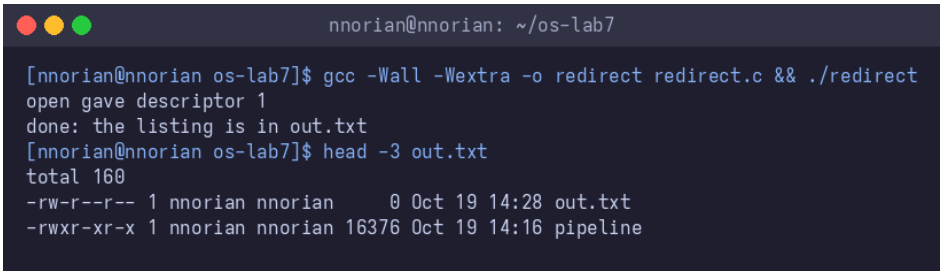


Figure 2.5 – Output redirection: ls -l > out.txt

Observe answers:

1. `open()` gave descriptor 1. This is because `close(1)` freed descriptor 1, and `open()` always returns the smallest available file descriptor number. Descriptors 0 (stdin) and 2 (stderr) were still open, so 1 was the smallest free number.

2. The listing from `ls` does not appear on the screen because `ls` writes to file descriptor 1, which now points to `out.txt` instead of the terminal. `ls` itself was not modified — it still calls `write(1, ...)` — but the meaning of descriptor 1 was changed by the redirect before exec.

3. To redirect errors (like `2> err.txt`), you would close descriptor 2 (stderr) before calling `open`. The `open` call would then return 2, and any output to stderr (including `perror` messages) would go to the file.

2.4 A pipe: ls /etc | wc -l

The program `pipeline.c` builds the pipeline `ls /etc | wc -l` by hand. It creates a pipe, forks twice: the first child closes its stdout and dups the pipe's write end to become

descriptor 1, then runs `ls`; the second child closes its stdin and dups the pipe's read end to become descriptor 0, then runs `wc`. The parent closes both pipe ends and waits for both children.

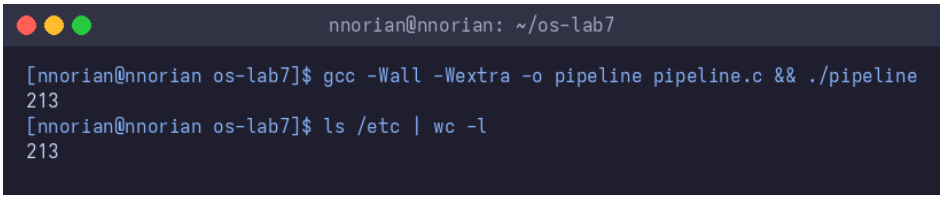


Figure 2.6 – The pipeline produces the same result as the shell

Both `./pipeline` and `ls /etc | wc -l` printed 213, confirming the hand-built pipeline works identically to the shell's.

Breaking the pipeline. When the parent's `close(p[1])` is commented out, the program hangs (Figure 2.7):

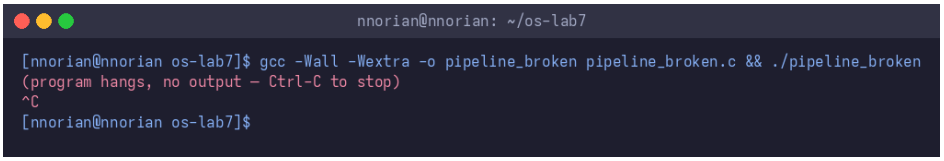


Figure 2.7 – The broken pipeline hangs and must be stopped with Ctrl-C

Observe answers:

1. Yes, `./pipeline` and `ls /etc | wc -l` gave the same number: 213.

2. After the change, `wc` is the child that never ends. `ls` finishes normally — it writes all filenames into the pipe and exits.

3. `wc` is waiting for EOF on its stdin (the read end of the pipe). EOF only arrives when all file descriptors that refer to the write end of the pipe are closed. The parent still holds `p[1]` open, so `wc`'s `read()` never returns 0 (EOF), and `wc` hangs forever waiting for more input.

3 The same calls, inside xv6 (Laboratory 8)

This chapter corresponds to Laboratory work 8. The same fork + exec + wait pattern was ported to xv6, and a new `pingpong` program was written using pipes.

3.1 run, inside xv6

The program `user/run.c` implements the same three steps as `run.c` from Lab 7, but uses xv6's system calls instead of glibc wrappers. The child becomes `echo` (instead of `ls` on Linux) to demonstrate exec.

Variant 1: echo hello from the child (Figure 3.1).

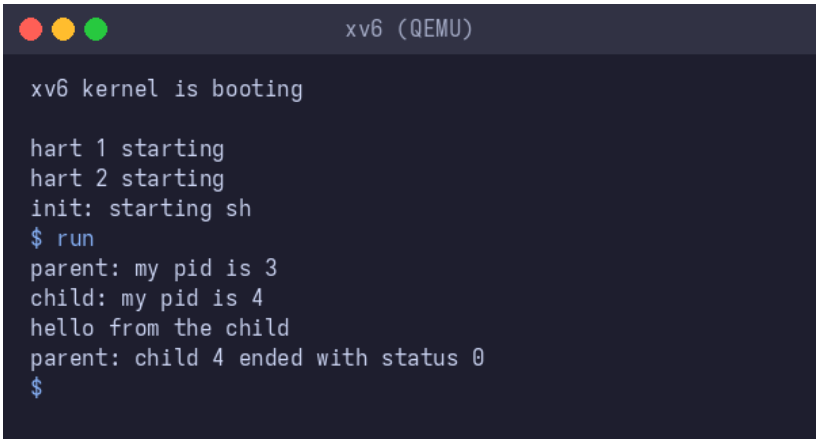


Figure 3.1 – run inside xv6 executing echo

Variant 2: ls (Figure 3.2).

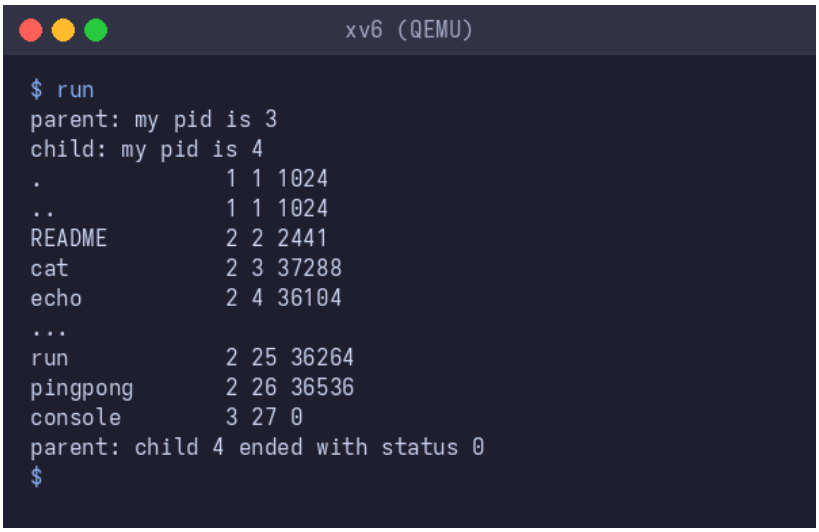


Figure 3.2 – run inside xv6 executing ls

Variant 3: nosuchprog (Figure 3.3).



Figure 3.3 – run inside xv6 with a nonexistent program

Observe answers:

1. In xv6, `run.c` uses `fork()`, `exec()`, and `wait()`. On Linux, `run.c` uses `fork()`, `execlp()`, and `waitpid()`. The core three-step pattern is the same: create a child (fork), replace its image (exec), and wait for it (wait). The differences are cosmetic: xv6 uses `exec()` with an `argv` array while Linux uses `execlp()` which searches PATH; xv6 uses `wait()` with a pointer to status while Linux uses `waitpid()` with more options.

2. `echo` prints "hello from the child". The code of `run.c` sets up the args array but does not print that text — `exec("echo", args)` replaces the child's memory with the echo program, and it is `echo`'s `main()` that iterates over its arguments and prints them.

3. The status is 1 in xv6 and 127 on Linux because the exit code after a failed exec is different in each program. In the xv6 version, the code after exec says `exit(1)`, while in the Linux version it says `exit(127)`. The convention of 127 for "command not found" comes from POSIX shells; xv6 has no such convention.

3.2 pingpong

The program `user/pingpong.c` demonstrates two-way communication between a parent and child process using two pipes: `p2c` (parent-to-child) and `c2p` (child-to-parent). The parent writes one byte into `p2c[1]`. The child reads it from `p2c[0]`, prints "received ping", writes the byte into `c2p[1]`, and exits. The parent reads the byte back from `c2p[0]`, prints "received pong", and waits for the child.

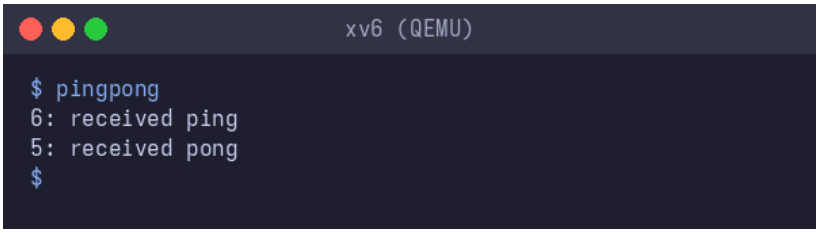


Figure 3.4 – pingpong inside xv6

The output shows PID 6 (child) received ping first, then PID 5 (parent) received pong — confirming the byte traveled parent → child → parent.

Observe answers:

1. Two pipes are needed because a pipe is one-way: data flows from the write end to the read end. If both parent and child wrote to and read from the same pipe, the parent could read back its own byte before the child gets it, creating a race condition. With two separate pipes, each direction has its own channel: `p2c` carries the ping (parent → child) and `c2p` carries the pong (child → parent).

2. The `read(c2p[0], &b, 1)` call makes the parent wait. `read()` on a pipe blocks until data is available. The parent calls `write()` to send the byte, then immediately calls `read()` on `c2p[0]`. Since the child hasn't written to `c2p[1]` yet, the parent's `read()` blocks until the child reads the byte, prints "received ping", and writes the byte back to `c2p[1]`, at which point the parent's `read()` returns with the data.

Conclusions

This laboratory work demonstrated the fundamental mechanisms that every Unix shell relies on: the fork-exec-wait pattern for running programs, file descriptor manipulation for redirection, and pipes for connecting processes.

Laboratory 7 showed these mechanisms from the outside on Linux. The `run.c` program made visible the three steps of process creation: `fork` duplicates the process, `exec` replaces the child's image with a new program, and `wait` collects the exit status. The zombie experiment showed what happens when the parent delays the wait call — the child becomes a defunct process visible in `ps` with state Z. The `redirect.c` program showed how the shell redirects output: by closing descriptor 1 and opening a file, the new file inherits descriptor 1 and all writes to stdout go to the file. The `pipeline.c` program showed how pipes work and, crucially, why every file descriptor that refers to the write end of a pipe must be closed — without this, the reader never gets EOF and hangs forever.

Laboratory 8 moved the same programs inside xv6. The `run.c` port confirmed that the three-step pattern (`fork`, `exec`, `wait`) is the same in xv6 as in Linux, with only surface differences in the API. The `pingpong` program demonstrated two-way IPC through pipes, and its observe answers highlighted two non-obvious properties: pipes are one-directional (requiring two for bidirectional communication) and `read()` on an empty pipe blocks (providing synchronization without explicit locks).

Together, the two labs show that the core process and I/O abstractions of Unix are stable across implementations: the same small set of system calls (`fork`, `exec`, `wait`, `pipe`, `read`, `write`, `close`, `open`, `dup`) is sufficient to build a shell, redirect output, and compose programs through pipes.

Bibliography

1) Operating Systems, FCIM / FAF, UTM. Laboratory guide 7 "Three Small Programs (Linux)" and Laboratory guide 8 "The Same Calls Inside Your Own OS (xv6)", 2026 –2027.
2) Cox R., Kaashoek F., Morris R. xv6: a simple, Unix-like teaching operating system (RISC-V version). MIT PDOS, https://pdos.csail.mit.edu/6.1810/.
3) MIT PDOS. xv6-riscv source repository, https://github.com/mit-pdos/xv6-riscv.
4) The Linux man-pages project. fork(2), exec(3), wait(2), pipe(2), dup(2), open(2), https://man7.org/linux/man-pages/.